

Developing a Provenance Monitor for Genomic Data with R Shiny

Oliwia Wójcicka

April 27, 2026

1 Introduction

The objective of this project was to develop an interactive R Shiny dashboard for monitoring provenance records generated during genomic data processing. In high-throughput sequencing environments, reproducibility and data integrity are critical. Each processing activity generates a JSON-LD provenance record based on the PROV-O style structure.

The application developed in this assignment transforms complex nested JSON-LD records into a readable dashboard. The dashboard allows laboratory managers to monitor system health, processing efficiency, and data throughput. It also allows users to inspect the full provenance schema of each selected record.

The main workflow of the project is:

```
100 JSON-LD files -> MongoDB database -> R Shiny dashboard
```

2 Input Data and JSON-LD Generation

The assignment required 100 JSON provenance records. Since real laboratory logs were not provided, 100 synthetic JSON-LD files were generated using an R script based on the schema shown in the assignment description. ChatGPT was used as an assistance tool to help design and prepare the R script for generating these synthetic JSON-LD files.

The generated records were used only for testing and demonstrating the functionality of the MongoDB import process and the Shiny dashboard. Each generated file represents one genomic data processing activity. The structure of the generated files follows the provenance schema provided in the assignment description.

Each JSON-LD record contains the following main fields:

- `@context`, `@id`, and `@type` to describe the provenance activity.
- `startTime` and `endTime` to calculate the processing duration.
- `executionNode` to identify the machine used for processing.
- `wasAssociatedWith` to store software agents and user information.
- `generated` to store SHA256 verification, Seqfu validation, FASTQ file count, and total file size.

The execution node names such as `node01`, `node02`, `node03`, `node04`, and `cresselia` come from the synthetic data generation script. They were included because the assignment requires analysis of execution nodes and their processing performance.

The JSON files were saved in the project folder:

```
data/json/
```

The generated files are named:

```
provenance_001.json
provenance_002.json
...
provenance_100.json
```

3 MongoDB Import

After generating the 100 JSON files, they were imported into a local MongoDB database using the R package `mongolite`. The database name used in this project is:

```
genomics
```

The collection name is:

```
provenance_logs
```

The import script performs the following steps:

1. Connects to the local MongoDB server.
2. Reads all JSON files from the `data/json/` folder.
3. Adds useful summary fields for the dashboard, such as duration, SHA256 status, Seqfu status, total size in GB, and file count.
4. Inserts each full JSON-LD record into MongoDB.

The following R code was used to connect to MongoDB:

```
mongo_conn <- mongo(
  collection = "provenance_logs",
  db = "genomics",
  url = "mongodb://localhost:27017"
)
```

After running the import script, the number of records in MongoDB was checked using:

```
mongo_conn$count()
```

The expected result was:

```
100
```

This confirms that all 100 JSON-LD provenance files were successfully imported into the MongoDB collection.

4 Shiny Dashboard Functionality

The web application was implemented using R Shiny and `shinydashboard`. The application connects directly to MongoDB and reads the imported provenance records from the `provenance_logs` collection.

The dashboard contains a sidebar layout with five main sections:

- **System Health** – monitors successful and failed integrity checks.
- **Efficiency & Nodes** – analyzes processing time by execution node.
- **Throughput** – summarizes the total amount of genomic data processed.
- **Provenance Records** – displays a searchable and filterable table of records.
- **Full JSON-LD Schema** – displays the complete provenance schema for the selected record.

4.1 System Health

The System Health section provides a quick overview of data integrity. It includes value boxes showing:

- Total provenance records in MongoDB.
- Number of records with failed SHA256 or Seqfu checks.
- Number of records with successful integrity checks.

It also includes a dynamic bar plot showing failed integrity checks by execution node. This helps identify whether failures are concentrated on a specific execution node.

4.2 Efficiency and Execution Nodes

The Efficiency section evaluates processing performance. The application calculates the duration of each activity using the difference between `startTime` and `endTime`. The dashboard displays:

- Average processing time by execution node.
- A table of the top 10 slowest processing records.

This allows users to identify slower execution nodes and unusually long processing activities.

4.3 Throughput

The Throughput section focuses on the amount of data moved to storage. The dashboard displays:

- Total gigabytes processed.
- Average gigabytes per provenance record.
- Total number of FASTQ files.
- Total data volume processed by each execution node.

The total size in bytes from the JSON-LD field `totalSizeBytes` is converted into gigabytes for easier interpretation.

4.4 Reactive Inputs and Dynamic Outputs

The application includes reactive inputs in the Provenance Records section. Users can filter the table by:

- Execution node.
- Integrity check status: all records, only failed records, or only successful records.

The dashboard outputs update dynamically when filters are changed. These dynamic outputs include value boxes, plots, and data tables.

5 Full Provenance Schema Display

One important requirement was to implement a feature where clicking a row in the table displays the full Provenance schema. This was implemented using the selected row from the DT table.

When a user selects a row in the **Provenance Records** table, the application retrieves the corresponding full JSON-LD document from MongoDB. The selected record is identified using its `record_id`. The complete JSON-LD record is then displayed in the **Full JSON-LD Schema** tab.

The relevant logic is:

```
selected <- input$records_table_rows_selected

if (length(selected) > 0) {
  data <- filtered_data()
  selected_id <- data$record_id[selected]
  query <- sprintf('{"record_id": "%s"}', selected_id)
  full_record <- mongo_conn$find(query, fields = '{"_id": 0}')
  cat(toJSON(full_record, pretty = TRUE, auto_unbox = TRUE))
}
```

This feature allows users to move from a summarized table view to the complete provenance metadata for a specific genomic processing activity.

6 Project Structure

The final project structure is shown below:

```
assignment3_data/
|-- app.R
|-- assignment3_data.Rproj
|-- data/
|   |-- json/
|   |   |-- provenance_001.json
|   |   |-- provenance_002.json
|   |   |-- ...
|   |   |-- provenance_100.json
|-- scripts/
|   |-- generate_json_files.R
|   |-- import_to_mongodb.R
```

The project is uploaded to a public GitHub repository:

<https://github.com/oliwiawojcicka/DataScience-in-biomedicine>.

7 How to Run the Application

To run the complete workflow, MongoDB must first be installed and running locally. Then the following commands can be executed in RStudio:

```
source("scripts/generate_json_files.R")
source("scripts/import_to_mongodb.R")
runApp()
```

The first command generates the 100 JSON-LD files. The second command imports them into MongoDB. The third command starts the Shiny dashboard.

If the JSON files have already been generated, the first command does not need to be repeated. In that case, the project can be run using:

```
source("scripts/import_to_mongodb.R")
runApp()
```

8 Conclusion

This project successfully implements a provenance monitoring dashboard for genomic data processing using synthetically generated JSON-LD records. The solution generates 100 JSON-LD provenance files, imports them into MongoDB, and displays them through an interactive R Shiny web application.

The application meets the main assignment requirements by combining JSON-LD provenance data, MongoDB storage, and an interactive Shiny interface with sidebar navigation, reactive inputs, dynamic outputs, and row-based inspection of complete provenance records.